CIS 4190/5190: Applied Machine Learning Fall 2026

Final Project

Release Date: October 13, 2026

1 Introduction

Welcome to the final project for CIS 4190/5190 Applied Machine Learning! This project is designed to be both a structured learning experience and an opportunity for exploration. The project specifications outlined here are **not intended to be fully prescriptive**. Instead, there will be open-ended components both to how you will go about achieving the objectives specified in the description (i.e., **basic components**), and also to the objectives that you set for yourselves beyond what is in the document (i.e., **exploratory components**). We hope that this project offers you the chance to deepen your understanding of machine learning concepts while also challenging you to explore new directions.

Each team (3 students) will work on one of two project directions:

A: Image2GPS: Predicting the camera location based on an input image

B: News Source Classification: Classifying the source of a news article from its link

1.1 Key Dates

Date	What is due
October 13, 2026	**Project released.**
October 22, 2026	**Leaderboards open** (Section 2). The leaderboard links will be posted on Ed.
October 31, 2026	**Checkpoint.** (1) Team formation and track selection submitted on Gradescope, including your Group ID, and (2) at least one submission to your track's leaderboard.
December 8, 2026	**Final submission.** Final model on the leaderboard; on Gradescope, the project report, made up of the 5-page written document (template attached separately) and the video demo (Section 1.7), plus your code and Hugging Face dataset link.

Read Section 2 (Submission and Leaderboard) before you start building. The leaderboard runs your code with a fixed input/output contract. If you write your data pipeline and model to that contract from day one, you will not have to rewrite them later, and you will be ready to submit when the leaderboards open on **October 22, 2026**. Your first leaderboard submission can be the baseline model; its purpose is to confirm that your pipeline runs end to end on our backend.

1.2 Team Formation and Project Track Selection

After you form your team of 3 members, one member makes a group submission to the Gradescope team-formation assignment by **October 31, 2026**, stating:

1. Your team members. Add your teammates by typing their names in the group member selection bar when you submit.
2. Which of the two topics you choose (or another topic; see Section 1.8, Exceptions, below).
3. Your team's **Group ID**: a random 6-digit number. Generate it with any random number generator rather than picking one by hand, e.g. `python -c "import random; print(random.randint(100000, 999999))"`. Use exactly the same Group ID every time you submit to the leaderboard; this is how we match your leaderboard entries to your team. If two teams end up with the same number, we will ask the team that submitted it later to change theirs.

Each team member is expected to contribute substantially to the project, and every member's individual contributions will be documented in your report (Section 1.5).

1.3 Basic Components

The basic components of this project is to ensure that you experience and execute each stage of a standard machine learning pipeline, from data preparation through model development and rigorous evaluation, mirroring best practices in practical ML workflows:

1. **Data Collection**: Collect, curate, clean, and split your dataset, ensuring it is ready for modeling and internal evaluation.
2. **Model Design**: Build and iteratively improve your model architecture, making design choices based on empirical findings.
3. **Model Evaluation**: Evaluate your models using appropriate metrics, select the best one based on performance, and submit it to the leaderboard.

To ensure a shared starting point, the course team will:

1. Provide starter data and skeleton code for both project directions. You can use them as the baseline for your work if applicable.
2. Run a live leaderboard for each track on Hugging Face (Section 2), opening **October 22, 2026**. Each team must submit at least one entry by **October 31, 2026**, and may keep submitting until the final deadline.
3. A portion of your project grade (about 15%) will be awarded for achieving at least a minimum benchmark score on the main problem (specifics and metrics defined in the project documents) and for surpassing a minimum level of performance on the problem. This benchmark part of the grade is measured on the leaderboard: your model should beat the baseline score shown on your track's leaderboard. Everything else is graded

from your final submission.

1.4 Exploratory Components

Here the intention is to try to accomplish something new and exciting that will become part of your "portfolio" on a resume. Lots of freedom, and points mainly for interestingness/creativity. Take risks! This is the part that's really yours! You should aim to create something of value to others, while also being of value for your own education e.g.

- **Code**: implementation of an existing algorithm in a new programming language or using new libraries or with higher efficiency, fixing bugs in an existing codebase
- **Technique**: e.g. improving a neural architecture
- **Analysis**: e.g. asking and exploring a new question to which the answer is not obvious
- **Teaching**: e.g. producing an explanatory blog post on a topic related to your project
- **Application**: e.g. applying a known algorithm to a new domain
- **Datasets**: e.g. collecting a new dataset for a topic

If in doubt, please post on Ed discussions or consult Office Hours!

1.5 Project Report

As part of the project, you will submit a report to document your approach, the experiments you conducted, and the insights you gained. Your report has two parts: a written document and a video demo (Section 1.7). Your report must address the following **basic components**:

- Describe the procedure and protocol for **data collection**, what considerations went into these, how you curated or cleaned up your datasets, how you split the datasets for internal evaluations (if you did), and what your final dataset ended up looking like.
- Describe the design considerations for the model, the iterative process through which you tried to improve those designs, and what your final **model design** ended up looking like.
- Describe the evaluation protocols and results (how you evaluated model iterations, what performance metrics you used, and how you chose models to submit to the leaderboard)
- Describe each team member's individual contributions to the project.

It must also address the **exploratory components**. Some directions (non-exhaustive) are listed below. You do not need to cover all of them: choose the ones that fit your project, and you may also present directions not listed here.

- **Code**: Summarize your motivation for code modifications, such as porting to a new

language or improving efficiency, and describe your approach, focusing on major implementation choices. Discuss specific challenges encountered and solutions implemented during development. Present benchmarks or comparisons showing the impact of your contributions on performance or usability.

- **Technique**: State the modeling or algorithmic innovation explored, including its theoretical motivation or inspiration from previous literature. Outline the main steps for implementation and validation of the new technique, and highlight any key design decisions or parameter choices. Briefly summarize results from experiments or ablation studies, comparing them to baseline performance.
- **Analysis**: Describe the research question or hypothesis you investigated, and detail the experimental design or analytical methods used to address it. Present your primary findings, supported by visualizations or summary statistics as appropriate. Discuss the insights gained and any implications or limitations that surfaced during your analysis.
- **Teaching**: Specify the teaching resource or artifact you developed, such as a blog post, demo, or presentation, and identify the intended audience. Explain your approach to breaking down complex concepts and making them accessible, including any visual or interactive elements used. Summarize feedback, audience engagement, or lessons learned from the process of creating educational materials.
- **Application**: Identify the real-world scenario or dataset where you applied your model, and note any modifications necessary to adapt your approach for this context. Provide a concise evaluation of model performance or utility in the new domain, mentioning both strengths and challenges encountered. Reflect on the broader applicability and relevance of your solution in practical settings.
- **Datasets**: Explain the motivation for collecting or curating new data and describe your sourcing and cleaning procedures. Give a brief summary of dataset characteristics, such as size, balance, labeling method, or notable attributes, using figures or tables if helpful. Note any difficulties with data quality and the steps you took to ensure the integrity and reproducibility of the dataset.
- **Comparison**: Analyze the pros and cons of State of the Art methods and how your work benefits from them or addresses their limitations. Use empirical results to compare your work with other SOTA to justify your claim. You don't have to outperform the SOTA methods.

1.6 Deliverables and Dates

By October 31, 2026:

- **Team formation and track selection** on Gradescope, including your Group ID.
- **At least one leaderboard submission** on your track's Hugging Face leaderboard (Section 2), which opens on October 22, 2026.

By December 8, 2026 (final submission):

- **Best-performing model** (on the leaderboard): your final `model.py`, `preprocess.py`, and (if needed) weights file, submitted as described in Section 2.
- **Project report** (on Gradescope): a 5-page written document that details your approach and findings with the structure described in Section 1.5, together with the video demo below.
- **Code** (on Gradescope): your full project code, either uploaded as a zip file with your report or as a GitHub link in your report.
- **Dataset** (link in your report): the dataset you collected and used in your experiments, uploaded as a Hugging Face Dataset (tutorial).
- **Video demo** (on Gradescope, part of your report): a video of at most 10 minutes presenting your project, uploaded as a video file, as described in Section 1.7.

1.7 Video Demo

New this year, each team also submits a video demo of their project. The video and the written document together make up your report.

- **Length:** at most **10 minutes**.
- **Presenters:** every team member must present some part of the video.
- **Content:** walk through the problem you tackled, how you collected and cleaned your data, your final model and its leaderboard result, and your exploratory component. Show your model making predictions on a few example inputs.
- **Submission:** upload the video file directly to Gradescope by the final deadline. Do not submit a link.

1.8 Exceptions

You are strongly encouraged to follow the standard guidelines for team size, project topics, and deliverables. Exceptions are rare and granted only for special cases mainly intended for: If you're performing research and can get your research-advising professor to sign off on an ML-related piece of the problem that you will lead with no help outside the team. To apply for an exception:

- Your advisor must agree to the external or expanded project responsibilities.
- Post a clearly labeled request ("Standard Project Exception Request [Your Name]") on Ed, outlining your intended topic and supporting documentation.
- Be aware that exceptions undergo extra scrutiny and receive less support from the teaching team.

2 Submission and Leaderboard

2.1 Submission Overview

We evaluate your code on the Hugging Face platform. Each track (Project A: Image2GPS, Project B: News Source Classification) has its own leaderboard. The leaderboard links will be posted on Ed when the leaderboards open on **October 22, 2026**.

For both project tracks, submit the following files:

- `model.py`: your model implementation (must be importable and instantiable).
- `preprocess.py`: your preprocessing function(s).
- `model.pt` or `model.pth` (optional): your trained weights; required if your model needs them to evaluate. Below, "`model.pt`" means either file.

When you submit, include:

- **Group ID:** your team's random 6-digit number (Section 1.2). Enter exactly the same Group ID every time. The leaderboard is not connected to Gradescope automatically; we match your leaderboard entries to your team using the Group ID you wrote in your Gradescope team-formation submission.
- **Alias:** the name shown for your team on the public leaderboard.

You can submit as many times as you like. You must submit at least once by **October 31, 2026**; your final model must be on the leaderboard by **December 8, 2026**. The leaderboard determines the benchmark part of your grade; everything else is graded from your final submission on Gradescope.

Each track has a student project folder with Python submission templates and local evaluation scripts that mirror our backend, plus starter data and a baseline model (for Project A, the starter photos in `train/`; for Project B, `url_train.csv`):

- Project A: Image2GPS project folder
- Project B: News project folder

2.2 Environment

Our backend environment includes the following:

- `numpy`, `pandas`, `torch==2.9.1`, `torchvision`, `scikit-learn`, `opencv-python`

Submissions run on **CPU only** (no GPU) and must finish within **7 minutes**. Train on whatever hardware you like, but make sure inference with your final model fits this limit on a CPU. We strongly encourage implementations in PyTorch. If you want to use another framework, leave an Ed post to let us know. If your code requires a library not in this list and no simple workaround exists, leave an Ed post as well; we will decide whether to add it to the environment.

2.3 Data Streams and Contracts

The backend runs your submitted `preprocess.py` and `model.py` with a strict I/O contract. Do not attempt to cheat the leaderboard. TAs will have access to your source code.

One input per row. `prepare_data` must return exactly one input for every row of the test CSV, in the same order. Do not filter, drop, or reorder rows inside `prepare_data`; do any cleaning of your training data separately. A submission that returns a different number of inputs than there are rows will fail.

2.3.1 Project A: Img2GPS

Project folder: Image2GPS project folder

Data. A CSV with images under our backend. The form is similar to the `train/` folder in the project folder above, whose CSV contains:

- Image path column (one of: `image_path`, `filepath`, `image`, `path`, `file_name`)
- Latitude column (one of: `Latitude`, `latitude`, `lat`)
- Longitude column (one of: `Longitude`, `longitude`, `lon`)

In the test CSV, the Latitude and Longitude columns are present but filled with zeros, since the true locations are hidden. Your code must not rely on them.

Preprocess. Your `preprocess.py` must expose:

```
def prepare_data(csv_path: str) -> (X, y)
```

- X: sequence/array/tensor of inputs suitable for `model.predict(batch)` or `model(batch)`. Images must be resized to 224×224.
- y: sequence/array/tensor of target pairs [lat, lon] in *degrees* (raw, not normalized).

If you use normalization, **hard code the stats** in your `model.py`.

Model. Your `model.py` must provide either:

- `get_model() -> model_instance`, or
- a class named `Model` or `IMG2GPS` that can be instantiated without arguments.

At inference, the backend calls `model.predict(batch)` if available; otherwise it calls `model(batch)`. It must return one [lat, lon] pair per input image, in degrees (raw, not normalized).

Weights. If you submit `model.pt`, we will load it into your model on CPU with `torch.load` and a robust `load_state_dict` routine. Ensure your checkpoint keys match your model's parameter names.

2.3.2 Project B: News Source Classification

Project folder: News project folder

Data. A CSV with a single `url` column, like `url_train.csv` in the project folder above. In the training data, the label (Fox News or NBC) comes from the website name in each link. In the test CSV, links are given **without the website name** (e.g., `/politics/some-headline`), so it contains no labels and your model only sees the link text. If you train on full links, strip the website name so your training inputs match the test inputs.

Preprocess. Your `preprocess.py` must expose:

```
def prepare_data(csv_path: str) -> (X, y)
```

- X: sequence/array/tensor of inputs suitable for your model.
- y: sequence of labels (strings or integer class ids). On the test CSV there are no labels to read, so y can be empty; the backend does not use it.

Model. Your `model.py` must provide either:

- `get_model() -> model_instance`, or
- a class named `Model` or `NewsClassifier` that can be instantiated without arguments.

At inference, the backend calls `model.predict(batch)` if present; otherwise it calls `model(batch)` and falls back to `argmax` over the final dimension if a tensor of logits is returned.

Weights. If you submit `model.pt`, the backend will load it as a state dict into your model before evaluation.

2.4 Evaluation Metrics

2.4.1 Project A: Img2GPS

We evaluate predictions strictly against the true *raw latitude/longitude* (degrees), which the backend keeps hidden. Let predicted pairs be $\hat{y}_i = (\hat{\phi}_i, \hat{\lambda}_i)$ and ground-truth be $y_i = (\phi_i, \lambda_i)$. **Leaderboard metric:** *Average Haversine distance (meters).* Lower is better.

$$d(a, b) = 2R \arcsin\left(\sqrt{\sin^2 \tfrac{\Delta\phi}{2} + \cos\phi_1 \cos\phi_2 \sin^2 \tfrac{\Delta\lambda}{2}}\right), \tag{1}$$

where $R = 6{,}371{,}000\,\text{m}$, $\Delta\phi = (\phi_2 - \phi_1)$ in radians, and $\Delta\lambda = (\lambda_2 - \lambda_1)$ in radians. We report $\frac{1}{N}\sum_i d(y_i, \hat{y}_i)$.

2.4.2 Project B: News Source Classification

Let predictions be $\hat{y}_i$ and ground-truth labels be y_i. We compute *accuracy*. If your model outputs integer ids while labels are strings (or vice versa), we apply a robust 2-class mapping when possible; otherwise we compare as strings.

2.5 End-to-End Flow

Project A (Img2GPS)

1. Read CSV (e.g., `test/metadata.csv`).
2. `preprocess.prepare_data(csv)` returns (X, y).
3. Instantiate `model` from `model.py`.
4. Load `model.pt` (if provided).
5. Run batched inference via `model.predict(batch)` or `model(batch)`.
6. Compare predictions to the true *raw* lat/lon, which the backend keeps; compute metrics.
7. Write JSON results and update leaderboard (backend).

Project B (News Source Classification)

1. Read CSV (e.g., `url_test.csv`).
2. `preprocess.prepare_data(csv)` returns (X, y).
3. Instantiate `model` from `model.py`.
4. Load `model.pt` (if provided).
5. Run batched inference via `model.predict(batch)` or `model(batch)`.
6. Compare predictions to the true labels, which the backend keeps (the `y` from `prepare_data` is not used); compute accuracy.
7. Write JSON results and update leaderboard (backend).

2.6 Local Sanity Checks (Strongly Recommended)

Each project folder includes a lightweight local evaluator that mimics the backend:

- Project A: `eval_project_a.py`
- Project B: `eval_project_b.py`

Run them with your paths to verify `preprocess.py`, `model.py`, and optionally `model.pt` before submission.

2.7 Packaging and Submission Tips

- Keep imports standard; avoid non-listed dependencies unless approved.
- Ensure `preprocess.prepare_data` and your model entry points exist and match the contracts above.
- If providing `model.pt`, ensure its keys match your model parameters.
- **Ensembles:** you still submit the same three files: `model.py`, `preprocess.py`, and a single `model.pt`. Put all sub-models and the averaging logic inside the one `Model` class in `model.py`, so the weights of every sub-model save into the one `model.pt`.
- **Runtime:** inference runs on CPU and must finish within 7 minutes. Test your final model on a CPU before submitting.
- **Project A:** outputs must be $[\text{lat}, \text{lon}]$ pairs in degrees and will be compared against raw labels.
- **Project B:** test links come without the website name (e.g., `/politics/some-headline`).
- Include your Hugging Face Dataset link in the report by the final deadline.

3 Image2GPS

Project folder: Image2GPS project folder

3.1 Problem Definition and Motivation

In this project, you will train a regression model to predict GPS coordinates from input images. The first step involves building a dataset that pairs images with their corresponding GPS labels. You will then use this data to train supervised computer vision models capable of estimating the location where each image was captured. This project will provide you with hands-on experience in standard computer vision pipelines and the opportunity to develop practical CV models for everyday use cases.

Problem Definition and Scope: The goal of this project is to build a computer vision model that could predict the GPS locations from any images taken from campus. To make it more feasible, we define the testing region to be on Penn's campus in between the rectangular region from 33rd and Walnut to 34th and Spruce st, as shown in Figure 1.

3.1.1 Performance metrics

In this section we give some details on how will the dataset and the model that you submit be tested on our end. The dataset testing is easier. Basically, whatever dataset you collect and submit should perform similar (i.e. provide non-trivial gains) on the baseline model to the toy dataset that we release. To rephrase, the baseline model should be good enough to provide non-trivial performance on your dataset. For the model that you submit, we will perform inference on the hidden test data and calculate the average Haversine distance between your predictions and the true GPS coordinates (Section 2.4.1). This is the mean distance in meters between where your model says each photo was taken and where it actually was, and it decides your position on the leaderboard (lower is better). Our test data will include cases that try to test your model for robustness against distribution shifts. So, you should try to think and come up with cases that your model might perform bad on and how can you mitigate them. Of course we will not test on images pointing towards the sky ;) Since, you will have the baseline model with you, we expect that the data you will collect will be of high quality and with proper testing you should be able to score well in the data collection component :)

3.2 Code for Training a Baseline Model

You can find our baseline, a naive method that just slightly outperforms trivial solutions (by only outputting the mean of latitude and longitude), in the `tested baseline` folder of the Image2GPS project folder: `model.py`, `preprocess.py`, `train.py`, and its trained weights `model.pth`.

Essentially, we resize the image to 224 by 224 (**you should also resize the image to 224 $\times$ 224**) and normalize it. More importantly, we normalize the outputs by calculating

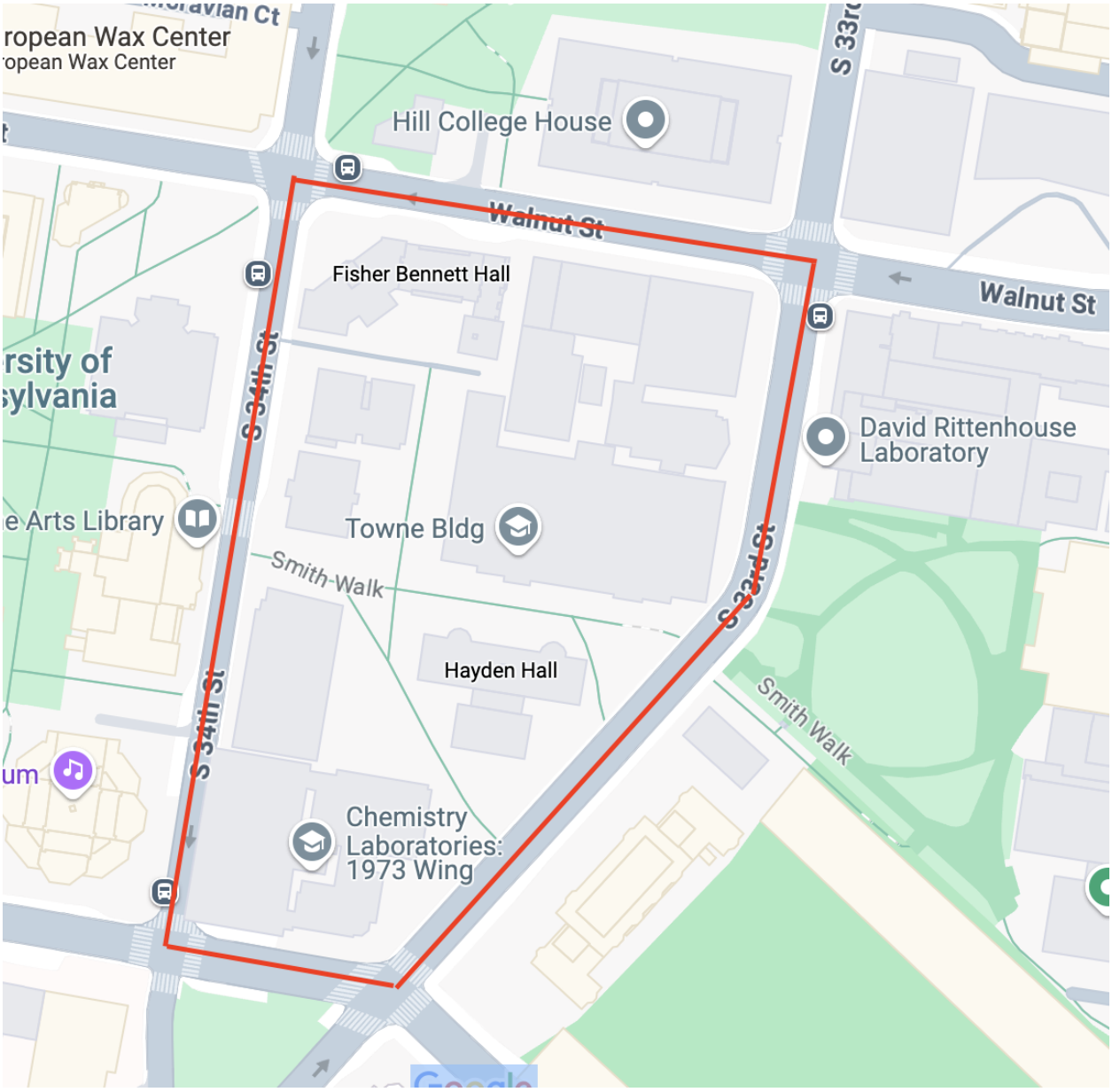


Figure 1: Test region for GPS prediction.

the mean and variance of the latitudes and longitudes (remember to scale it back after prediction).

We use a ResNet-18 as the backbone and modify the output of the last layer to have a dimension of two. We conduct full fine-tuning using MSE loss, the Adam optimizer with a learning rate of 0.001, and a scheduler that decreases the learning rate every few epochs. In total, we run 10-15 epochs to obtain baseline performance. Your model is expected to achieve performance at least as good as this baseline: beat the baseline score shown on the leaderboard.

A reminder: make sure to set the Colab runtime to connect with GPUs. Note that you are also likely to run out of GPU allocation if you are using a free account.

3.3 Other Miscellaneous Resources

3.3.1 Data Collection

In modern computer vision tasks, models are often trained on large-scale image datasets available online, such as COCO and ImageNet. However, assembling a dataset that effectively supports your specific tasks can be more challenging than it appears. In the initial stage of this project, you will collect an image dataset with GPS labels. You will learn how to gather high-quality images suitable for this task, ensure consistency across different data collectors, and account for factors that may affect data distribution such as lighting and weather.
We will implement a standardized data collection procedure. First, the test dataset will consist exclusively of images taken along walkways, so you won't need to cover every inch of the campus. Second, it's important to note that each GPS location can correspond to multiple views. To ensure comprehensive coverage, you should capture several photos from different viewpoints at the same exact location. During data collection, we usually stand at a point and rotate to take eight pictures from various angles. When taking pictures, avoid zooming in or out, and always hold your phone upright.

3.3.2 Post Process and Extracting GPS Location

You can easily access the GPS location of the image that you captured through the EXIF data of the image. For this you will need to enable settings from your smartphone that allow you to capture/store the EXIF data of the picture that you take. We will give suggested approach for iPhone and Android (Samsung). Note that this may slightly differ for your smartphone but the general idea is the same. For iPhone, you will need to go to (Settings)—>(Privacy)—>(Location Services) and make sure that the application for Camera is given the access. For Android, the default camera application will have settings. You should open settings from inside the default camera app and make sure that the Location/Location tags option is selected. Note, you should always make sure that there is EXIF data collected for the picture that you capture first before starting to collect more images. To help you with this, please find the ipynb notebook. The notebook will extract the EXIF data (specifically, the latitude and longitude) from the image and store it in a CSV file. Feel free to create

a copy of the notebook and change it to your requirements. This way you can create the training labels for the image that you capture.

3.3.3 Uploading Data and Model

Submit your model to the Image2GPS leaderboard (link posted on Ed when the leaderboards open on October 22, 2026) as `model.py`, `preprocess.py`, and (optionally) `model.pt` or `model.pth`, following the Project A contract in Section 2. Upload your collected images and labels as a Hugging Face Dataset and include its link in your report.

4 News Source Classification

Project folder: News project folder

4.1 Problem Definition and Motivation

In this topic, you will train a binary text classifier that predicts whether a news article comes from **Fox News** or **NBC News**, using only the article's link. At test time, each link is given without the website name (e.g., `/politics/some-headline`), so your model only sees the link text. We provide `url_train.csv`, which contains 4,800 article links from the two outlets, and a baseline model (a small neural network). Both are in the News project folder. Your task will be to build a dataset from these links (and any more you collect), process it, and experiment with various models to improve classification performance as best as you can. To show and summarize your improvement, in the final report, you will submit one or several line charts of your models' metrics, including metrics of the baseline model.

4.1.1 Performance metrics

We evaluate your model by its accuracy (Section 2.4.2) on a hidden test set of Fox News and NBC News article links, given without the website name. Your model will not have access to the test set when you are training/validating your model. In the sections below, we give you tips on how to clean the data, as well as how to do web scraping if you want to use article headlines as extra training data.

4.2 Baseline Model

The baseline is a small neural network, provided in the `tested baseline` folder inside the News project folder. Its score is shown on the leaderboard. Your model should beat the baseline score shown on the leaderboard.

4.3 Other Miscellaneous Resources

`url_train.csv` contains article links from Fox News and NBC News. To read it in Python, you can use the `pandas` library. Your submitted model only receives the link text, but you can also scrape each article's headline from its link, for example to build extra training data or to analyze what distinguishes the two outlets. For those new to web scraping, please follow the steps below. The Python libraries that are most helpful are `BeautifulSoup` and `request`. To web scrape, you identify the HTML tabs that contain the text that you want. In the example below, we are scrapping the headline which is in a `<h1>` tag with the class name `"headline speakable."` Here is a sample Python code to web scrape:

```
import requests
from bs4 import BeautifulSoup

url = https://www.foxnews.com/sports/juan-soto-sends-yankees-world-
   series-first-time-15-years # example website
```

```
5
6  response = requests.get(url)
7  if response.status_code != 200:
8    raise Exception(f"Failed to load page:
9    Status code {response.status_code}")
10
11 # Parse the HTML content with BeautifulSoup
12 soup = BeautifulSoup(response.text, "html.parser")
13
14 title = soup.find("h1", class_="headline speakable")
15
16 # title should be the following
17 # Juan Soto sends the Yankees to the World Series for the first time in
       15 years
```

You are also encouraged to webscrap and collect your own headlines from Fox News and NBC News.

4.3.1 Cleaning Process Suggestions

Whether you work with the link text or with scraped headlines, consider cleaning to make sure your data is ready for training. Some possible operations are listed, but they are not required or no guarantee of performance improvement. You are highly encouraged to explore on your own and incorporate more advanced techniques:

- **Data Cleaning**: Remove unwanted elements such as HTML tags, scripts, and special characters. Address unnecessary spaces, tabs, and newline characters to ensure consistency. Decide how to handle punctuation marks within the headlines.
- **Normalization**: Standardize the text by converting it to lowercase or uppercase. Remove common stopwords that may not add significant value. Apply stemming or lemmatization to reduce words to their base forms.
- **Handling Missing or Incomplete Data**: Detect any missing or incomplete headlines in your dataset. Choose methods to handle these gaps, such as removing incomplete entries or imputing missing values.
- **Consistency Checks**: Ensure all headlines follow a consistent format. Identify and address duplicate headlines to maintain the integrity of your dataset.
- **Data Validation**: Manually inspect a subset of processed headlines for quality assurance. Calculate metrics such as average headline length or word frequency distributions to understand your dataset better.
- **Documentation and Tracking**: Maintain detailed notes on the preprocessing steps applied. Use version control systems like Git to manage changes in your preprocessing workflows.

4.3.2 Uploading Data and Model

Submit your model to the News Source Classification leaderboard (link posted on Ed when the leaderboards open on October 22, 2026) as `model.py`, `preprocess.py`, and (optionally) `model.pt` or `model.pth`, following the Project B contract in Section 2. Upload the dataset you trained on (links, plus any headlines you scraped) as a Hugging Face Dataset and include its link in your report.

5 Frequently Asked Questions

These are questions students asked about last year's project, with the course staff's answers.

Gradescope has no field for our group members. Where do we enter them? When you make the team-formation submission on Gradescope, type your teammates' names in the group member selection bar.

Does the report need to cover every exploratory direction (code, analysis, teaching, ...)? No. Choose any of the listed directions, or present something not on the list. Five pages is not meant to cover them all.

Can we submit an ensemble that averages several models? Yes, but you still submit the same three files: `model.py`, `preprocess.py`, and a single `model.pt`. Put all sub-model definitions and the averaging logic inside the one `Model` class in `model.py`, so every sub-model's weights save into the one `model.pt` and the ensemble behaves as one large model.

(Img2GPS) Must input images be exactly 224×224? Yes. Resize images to 224×224 in your `preprocess.py`.